МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ ДЕРЖАВНИЙ ВИЩИЙ НАВЧАЛЬНИЙ ЗАКЛАД “УЖГОРОДСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ” ІНЖЕНЕРНО-ТЕХНІЧНИЙ ФАКУЛЬТЕТ КАФЕДРА КОМП’ЮТЕРНИХ СИСТЕМ ТА МЕРЕЖ

ЗВІТ

до лабораторної роботи №1

з дисципліни “Програмування мікроконтролерів”

Студента 2-го курсу

Спеціальності 123- “Комп’ютерна інженерія”

Скоробогатого Максима Максимовича

м. Ужгород – 2026 р.

ЛАБОРАТОРНА РОБОТА No3

Використання матричної клавіатури для введення інформації у інтегрованому середовищі PSoC® CreatorTM

Мета: Ознайомитися з принципами підключення та використання матричної клавіатури для введення інформації у мікроконтролерних системах на базі PSoC. Набути навичок розробки проєкту в інтегрованому середовищі PSoC® CreatorTM, налаштування апаратних компонентів та програмної обробки введених даних.

ПОРЯДОК ВИКОНАННЯ РОБОТИ

1. Копіюємо проєкт Лабораторної роботи No2.
2. З каталогу Cypress додаємо на схему три Двонаправлені піни (Digital Bidirectional Pin), які будемо використовувати в якості стовпців (Column) та чотири піни типу «Вхід» (Digital Input Pin), що будуть відповідати рядкам (Row) нашої матричної клавіатури (рис. 1).

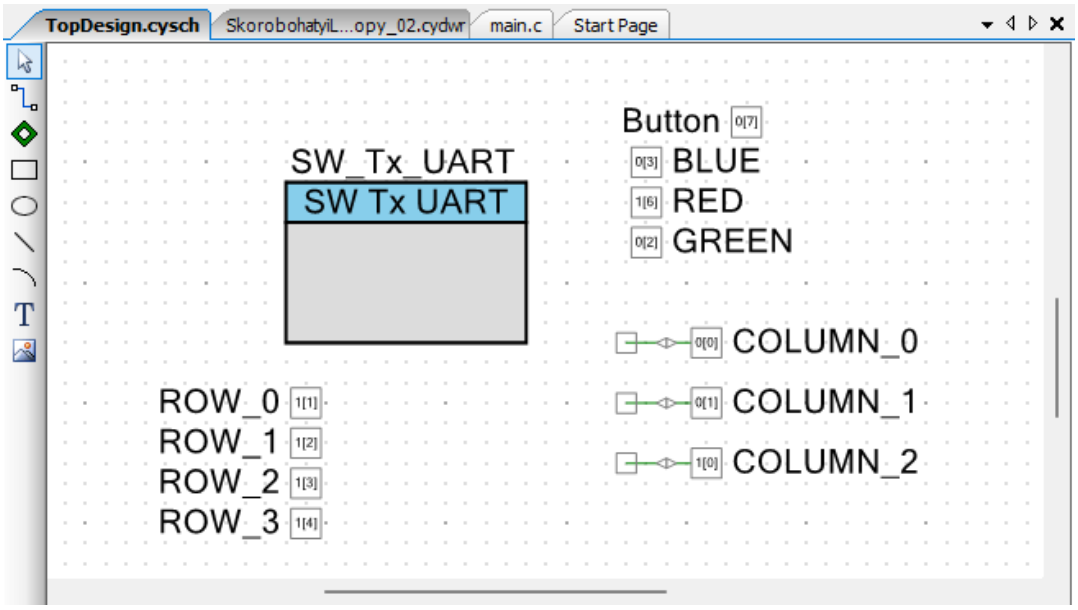

Рисунок 1 - Модифікація проєкту Лабораторної роботи No2

Після цього виконаємо налаштування Двонаправлених пінів Digital Bidirectional (рис. 2) та пінів, типу «Вхід» Digital Input (рис. 3):

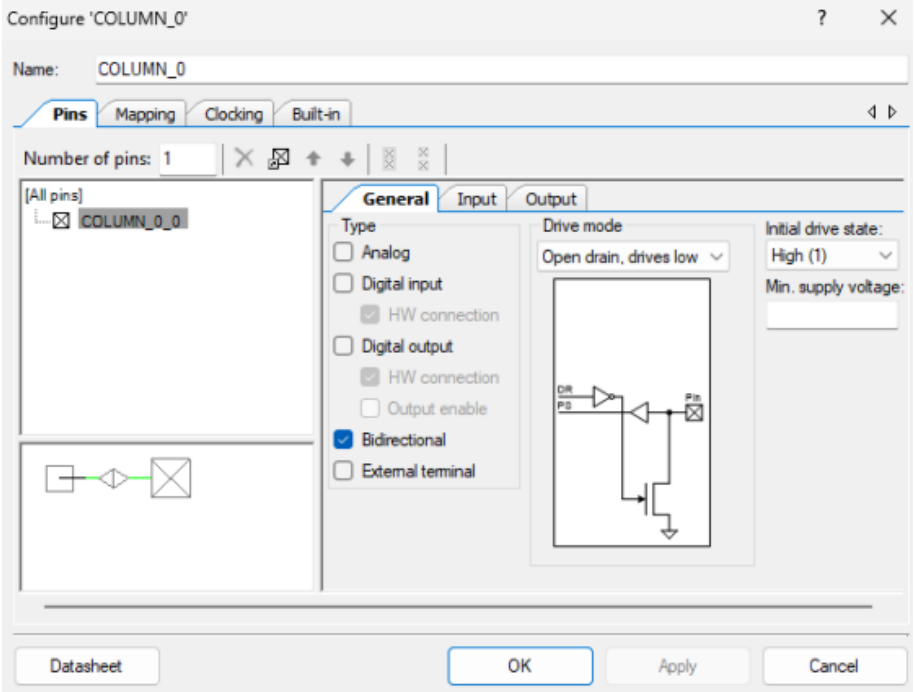

Рисунок 2 - Конфігурація стовпців

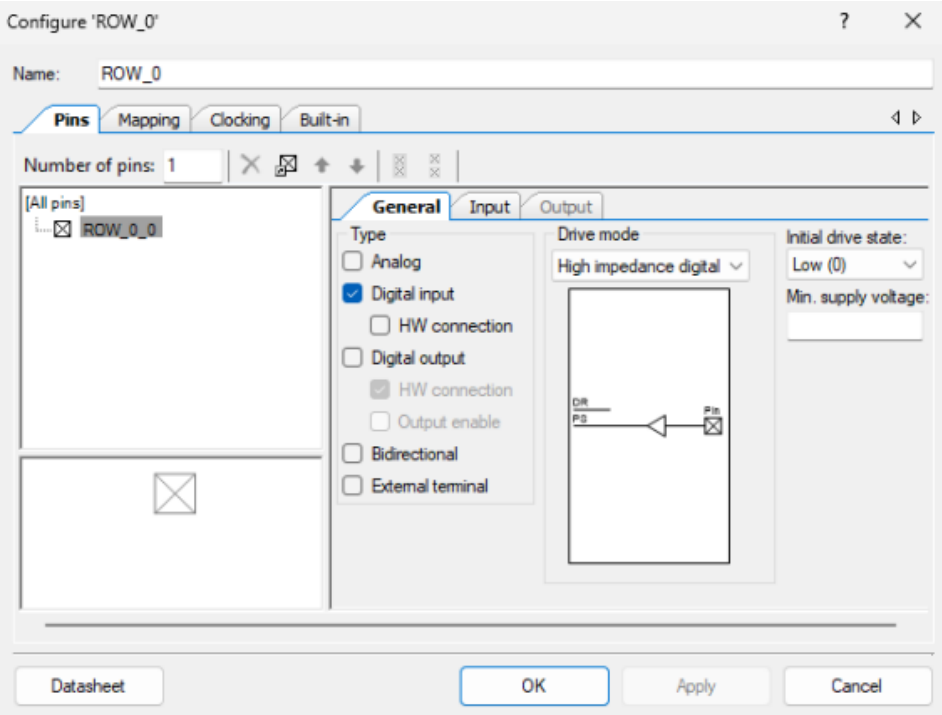

Рисунок 3 - Конфігурація рядків

3. Переходимо на вкладку Pins та підключаємо рядки та стовпці (рис. 4) в відповідності до схеми пінів на платі (рис. 5).

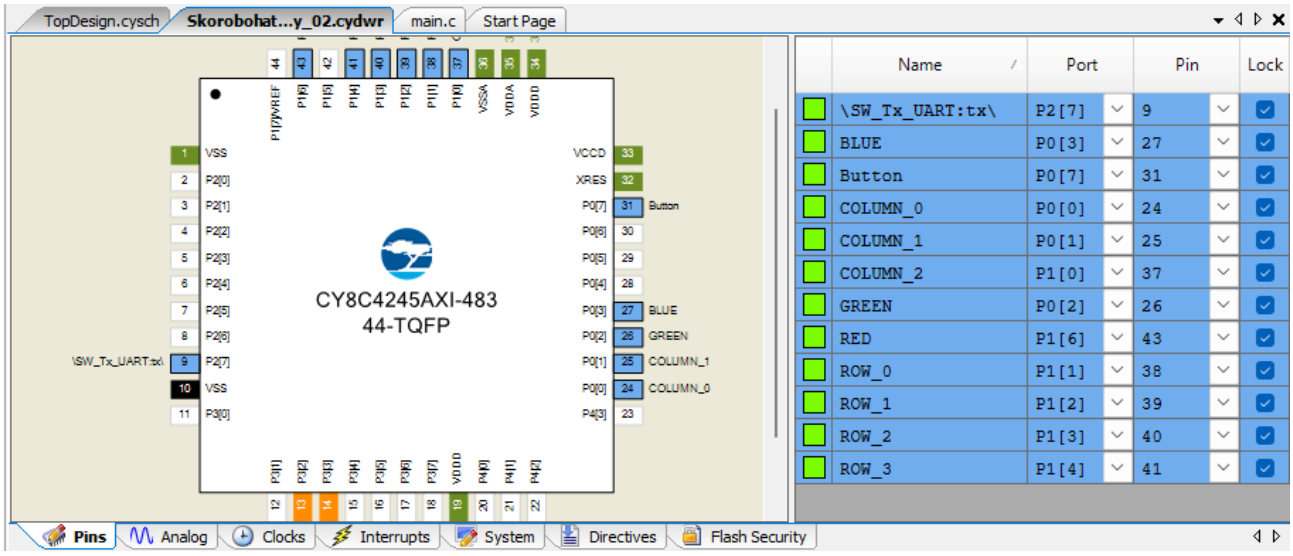

Рисунок 4 – Підключення пінів

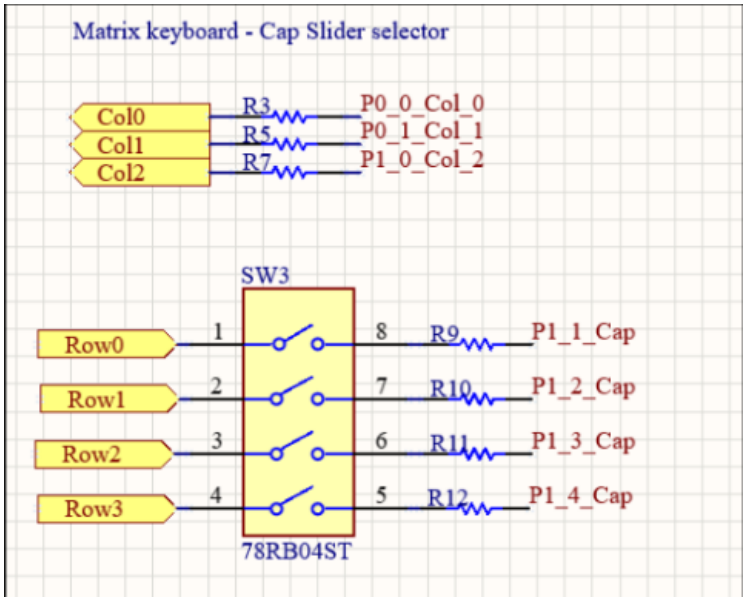

Рисунок 5 - Відповідність пінів на схемі

4. Для використання матричної клавіатури перемикаємо всі перемикачі SW3 в положення ON. Під час використання матричної клавіатури слайдер CapSence використовувати не вийде. (рис. 6).

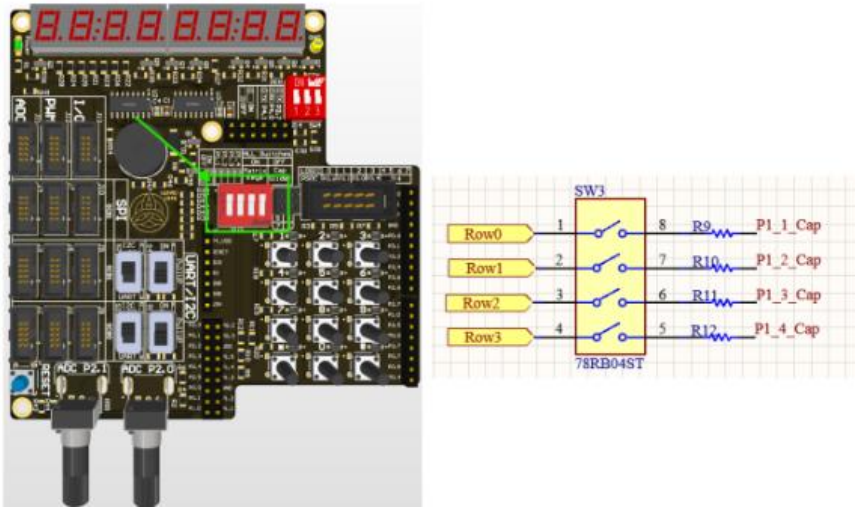

Рисунок 6 - Конфігурація перемикачів

5. Пишемо код для опрацювання матричної клавіатури:

1) Створюємо масиви вказівників на функції конфігурації, запису та читання піна.

2) Оголошуємо масив фактичних значень кнопок (матриця станів кнопок та масив відповідності).

3) Пишемо функцію ініціалізації, щоб перевести всі вихідні піни в непідключений стан (HIZ).

4) Пишемо функцію для опрацювання кнопок.

5) Додаємо ініціалізацію та вивід значень в main().

6) Компілюємо та запускаємо проєкт за допомоги програми Putty (рис 7.).

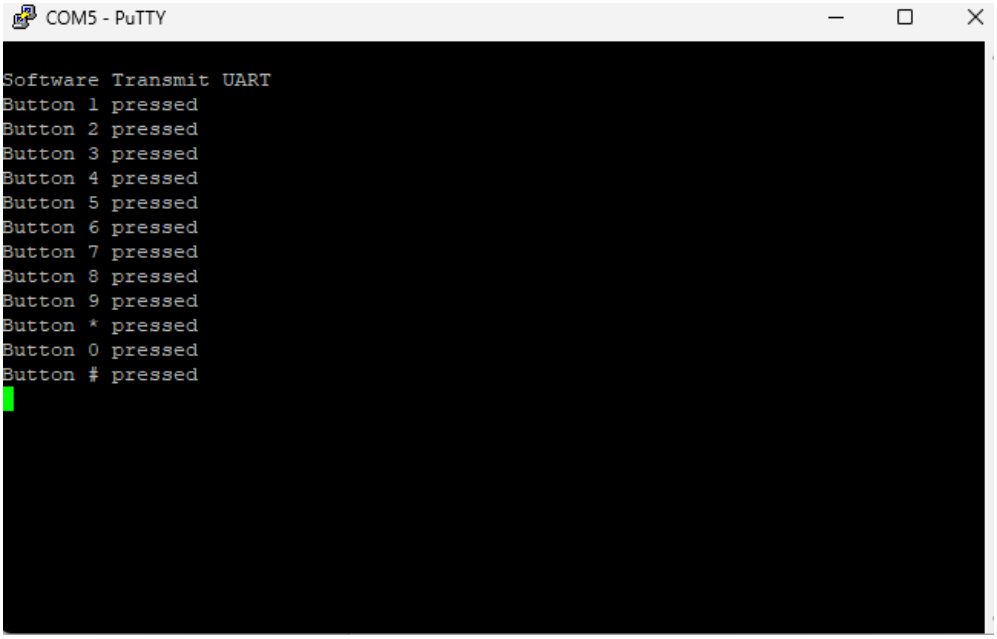

Рисунок 7 – Вивід програми

Лістинг коду

```
#include "project.h"

static void (*COLUMN_X_SetDriveMode[3]) (uint8_t mode) = {
    COLUMN_0_SetDriveMode,
    COLUMN_1_SetDriveMode,
    COLUMN_2_SetDriveMode
};

static void (*COLUMN_x_Write[3]) (uint8_t value) = {
    COLUMN_0_Write,
    COLUMN_1_Write,
    COLUMN_2_Write
};

static uint8 (*ROW_x_Read[4]) () = {
    ROW_0_Read,
    ROW_1_Read,
    ROW_2_Read,
    ROW_3_Read
};

static uint8_t keys[4][3] = {
    {1,  2,  3},
    {4,  5,  6},
    {7,  8,  9},
    {10, 0, 11}
};

static void initMatrix()
{
    for(int column_index = 0; column_index < 3; column_index++)
    {
        COLUMN_X_SetDriveMode[column_index](COLUMN_0_DM_DIG_HIZ);
    }
}

static void readMatrix()
{
    uint8_t row_counter = sizeof(ROW_x_Read)/sizeof(ROW_x_Read[0]);
    uint8_t column_counter = sizeof(COLUMN_x_Write)/sizeof(COLUMN_x_Write[0]);

    for(int column_index = 0; column_index < column_counter; column_index++)
    {
        COLUMN_X_SetDriveMode[column_index](COLUMN_0_DM_STRONG);
        COLUMN_x_Write[column_index](0);

        for(int row_index = 0; row_index < row_counter; row_index++)
        {
            keys[row_index][column_index] = ROW_x_Read[row_index]();
        }

        COLUMN_X_SetDriveMode[column_index](COLUMN_0_DM_DIG_HIZ);
    }
}

int main(void)
{
    CyGlobalIntEnable;

    SW_Tx_UART_Start();
    CyDelay(50);

    SW_Tx_UART_PutCRLF();
    SW_Tx_UART_PutString("Software Transmit UART");
    SW_Tx_UART_PutCRLF();

    initMatrix();

    uint8_t last_state = 0;

    for(;;)
    {
        readMatrix();
        if (keys[0][0]==1 && keys[0][1]==1 && keys[0][2]==1 &&
            keys[1][0]==1 && keys[1][1]==1 && keys[1][2]==1 &&
            keys[2][0]==1 && keys[2][1]==1 && keys[2][2]==1 &&
            keys[3][0]==1 && keys[3][1]==1 && keys[3][2]==1)
        {
            last_state = 0;
        }
```

```
        if (keys[0][0] == 0 && last_state != 1) {
            last_state = 1;
            SW_Tx_UART_PutString("Button 1 pressed");
            SW_Tx_UART_PutCRLF();
        }
        if (keys[0][1] == 0 && last_state != 2) {
            last_state = 2;
            SW_Tx_UART_PutString("Button 2 pressed");
            SW_Tx_UART_PutCRLF();
        }
        if (keys[0][2] == 0 && last_state != 3) {
            last_state = 3;
            SW_Tx_UART_PutString("Button 3 pressed");
            SW_Tx_UART_PutCRLF();
        }

        if (keys[1][0] == 0 && last_state != 4) {
            last_state = 4;
            SW_Tx_UART_PutString("Button 4 pressed");
            SW_Tx_UART_PutCRLF();
        }
        if (keys[1][1] == 0 && last_state != 5) {
            last_state = 5;
            SW_Tx_UART_PutString("Button 5 pressed");
            SW_Tx_UART_PutCRLF();
        }
        if (keys[1][2] == 0 && last_state != 6) {
            last_state = 6;
            SW_Tx_UART_PutString("Button 6 pressed");
            SW_Tx_UART_PutCRLF();
        }

        if (keys[2][0] == 0 && last_state != 7) {
            last_state = 7;
            SW_Tx_UART_PutString("Button 7 pressed");
            SW_Tx_UART_PutCRLF();
        }
        if (keys[2][1] == 0 && last_state != 8) {
            last_state = 8;
            SW_Tx_UART_PutString("Button 8 pressed");
            SW_Tx_UART_PutCRLF();
        }
        if (keys[2][2] == 0 && last_state != 9) {
            last_state = 9;
            SW_Tx_UART_PutString("Button 9 pressed");
            SW_Tx_UART_PutCRLF();
        }

        if (keys[3][0] == 0 && last_state != 10) {
            last_state = 10;
            SW_Tx_UART_PutString("Button * pressed");
            SW_Tx_UART_PutCRLF();
        }
        if (keys[3][1] == 0 && last_state != 11) {
            last_state = 11;
            SW_Tx_UART_PutString("Button 0 pressed");
            SW_Tx_UART_PutCRLF();
        }
        if (keys[3][2] == 0 && last_state != 12) {
            last_state = 12;
            SW_Tx_UART_PutString("Button # pressed");
            SW_Tx_UART_PutCRLF();
        }

        CyDelay(20);
    }
}
```

7) Модифікуємо main() з функцією для виводу стану матриці (рис.8).

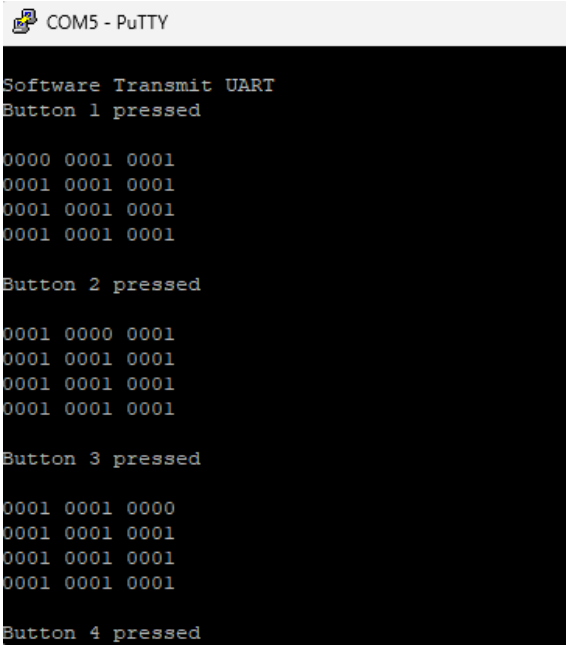

Рисунок 8 – Вивід програми

Лістинг коду

```
void printMatrix()
{
    SW_Tx_UART_PutCRLF();
    for (int row_index = 0; row_index < 4; row_index++)
    {
        for (int column_index = 0; column_index < 3; column_index++)
        {
            SW_Tx_UART_PutHexInt(keys[row_index][column_index]);
            SW_Tx_UART_PutString(" ");
        }
        SW_Tx_UART_PutCRLF();
    }
    SW_Tx_UART_PutCRLF();
}
```

8) Додаємо вивід натиснутої кнопки в термінал, виводиться подія «Button pressed» та подія «Button released» (рис.9).

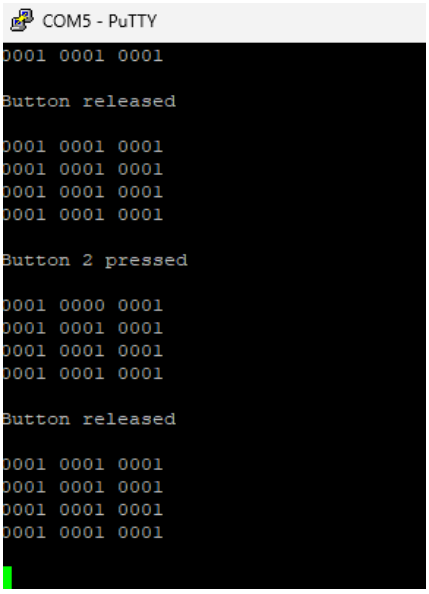

Рисунок 9 – Вивід програми

Лістинг коду

```
        readMatrix();
        current_pressed = 0;

        for (int r = 0; r < 4; r++)
        {
            for (int c = 0; c < 3; c++)
            {
                if (keys[r][c] == 0)
                {
                    current_pressed = key_map[r][c];
                }
            }
        }

        if (current_pressed != last_pressed)
        {
            if (current_pressed == 0)
            {
                SW_Tx_UART_PutString("Button released\r\n");
                setLEDColor(1, 1, 1);
                printMatrix();
            }
            else
            {
                switch (current_pressed)
                {
                    case 1:
                        SW_Tx_UART_PutString("Button 1 pressed\r\n");
                        setLEDColor(0, 1, 1);
                        break;
                    case 2:
                        SW_Tx_UART_PutString("Button 2 pressed\r\n");
                        setLEDColor(1, 0, 1);
                        break;
                    // ... інші case ...
                }
                printMatrix();
            }

            last_pressed = current_pressed;
        }
```

ІНДИВІДУАЛЬНІ ЗАВДАННЯ

ЗАВДАННЯ 1-2

```
#include "project.h"

void setLEDColor(uint8_t r, uint8_t g, uint8_t b) {
    RED_Write(r);
    GREEN_Write(g);
    BLUE_Write(b);
}

static void (*COLUMN_X_SetDriveMode[3]) (uint8_t mode) = {
    COLUMN_0_SetDriveMode,
    COLUMN_1_SetDriveMode,
    COLUMN_2_SetDriveMode
};

static void (*COLUMN_x_Write[3]) (uint8_t value) = {
    COLUMN_0_Write,
    COLUMN_1_Write,
    COLUMN_2_Write
};

static uint8 (*ROW_x_Read[4]) () = {
    ROW_0_Read,
    ROW_1_Read,
    ROW_2_Read,
    ROW_3_Read
};

static uint8_t keys[4][3];
```

```c
static const uint8_t key_map[4][3] = {
    {1,  2,  3},
    {4,  5,  6},
    {7,  8,  9},
    {10, 11, 12}
};

static void initMatrix()
{
    for(int column_index = 0; column_index < 3; column_index++)
    {
        COLUMN_X_SetDriveMode[column_index](COLUMN_0_DM_DIG_HIZ);
    }
}

static void readMatrix()
{
    for(int column_index = 0; column_index < 3; column_index++)
    {
        COLUMN_X_SetDriveMode[column_index](COLUMN_0_DM_STRONG);
        COLUMN_x_Write[column_index](0);

        for(int row_index = 0; row_index < 4; row_index++)
        {
            keys[row_index][column_index] = ROW_x_Read[row_index]();
        }

        COLUMN_X_SetDriveMode[column_index](COLUMN_0_DM_DIG_HIZ);
    }
}

void printMatrix()
{
    SW_Tx_UART_PutCRLF();
    for (int row_index = 0; row_index < 4; row_index++)
    {
        for (int column_index = 0; column_index < 3; column_index++)
        {
            SW_Tx_UART_PutHexInt(keys[row_index][column_index]);
            SW_Tx_UART_PutString(" ");
        }
        SW_Tx_UART_PutCRLF();
    }
    SW_Tx_UART_PutCRLF();
}

int main(void)
{
    CyGlobalIntEnable;

    SW_Tx_UART_Start();
    CyDelay(50);

    SW_Tx_UART_PutCRLF();
    SW_Tx_UART_PutString("Software Transmit UART");
    SW_Tx_UART_PutCRLF();

    initMatrix();
    setLEDColor(0, 0, 0);

    uint8_t last_pressed = 0;
    uint8_t current_pressed = 0;

    for(;;)
    {
        readMatrix();
        current_pressed = 0;

        for (int r = 0; r < 4; r++)
        {
            for (int c = 0; c < 3; c++)
            {
                if (keys[r][c] == 0)
```

```
                {
                    current_pressed = key_map[r][c];
                }
            }
        }

        if (current_pressed != last_pressed)
        {
            if (current_pressed == 0)
            {
                SW_Tx_UART_PutString("Button released\r\n");
                setLEDColor(1, 1, 1);
                printMatrix();
            }
            else
            {
                switch (current_pressed)
                {
                    case 1:
                        SW_Tx_UART_PutString("Button 1 pressed\r\n");
                        setLEDColor(0, 1, 1);
                        break;
                    case 2:
                        SW_Tx_UART_PutString("Button 2 pressed\r\n");
                        setLEDColor(1, 0, 1);
                        break;
                    case 3:
                        SW_Tx_UART_PutString("Button 3 pressed\r\n");
                        setLEDColor(1, 1, 0);
                        break;
                    case 4:
                        SW_Tx_UART_PutString("Button 4 pressed\r\n");
                        setLEDColor(0, 0, 1);
                        break;
                    case 5:
                        SW_Tx_UART_PutString("Button 5 pressed\r\n");
                        setLEDColor(0, 1, 0);
                        break;
                    case 6:
                        SW_Tx_UART_PutString("Button 6 pressed\r\n");
                        setLEDColor(1, 0, 0);
                        break;
                    case 7:
                        SW_Tx_UART_PutString("Button 7 pressed\r\n");
                        setLEDColor(0, 1, 1);
                        break;
                    case 8:
                        SW_Tx_UART_PutString("Button 8 pressed\r\n");
                        setLEDColor(1, 0, 1);
                        break;
                    case 9:
                        SW_Tx_UART_PutString("Button 9 pressed\r\n");
                        setLEDColor(1, 1, 0);
                        break;
                    case 10:
                        SW_Tx_UART_PutString("Button * pressed\r\n");
                        setLEDColor(0, 0, 1);
                        break;
                    case 11:
                        SW_Tx_UART_PutString("Button 0 pressed\r\n");
                        setLEDColor(0, 1, 0);
                        break;
                    case 12:
                        SW_Tx_UART_PutString("Button # pressed\r\n");
                        setLEDColor(1, 0, 0);
                        break;
                }
                printMatrix();
            }
            last_pressed = current_pressed;
        }
        CyDelay(20);
    }
}
```

ЗАВДАННЯ 3

COM5 - PuTTY

```
Button 1 pressed

0000 0001 0001
0001 0001 0001
0001 0001 0001
0001 0001 0001

Button released

0001 0001 0001
0001 0001 0001
0001 0001 0001
0001 0001 0001

Button 2 pressed

0001 0000 0001
0001 0001 0001
0001 0001 0001
0001 0001 0001

Button released

0001 0001 0001
0001 0001 0001
0001 0001 0001
0001 0001 0001

Button 3 pressed

0001 0001 0000
0001 0001 0001
0001 0001 0001
0001 0001 0001


=== Access allowed ===

Button released

0001 0001 0001
0001 0001 0001
0001 0001 0001
```

```
Button 4 pressed

0001 0001 0001
0000 0001 0001
0001 0001 0001
0001 0001 0001

Button released

0001 0001 0001
0001 0001 0001
0001 0001 0001
0001 0001 0001

Button 5 pressed

0001 0001 0001
0001 0000 0001
0001 0001 0001
0001 0001 0001

Button released

0001 0001 0001
0001 0001 0001
0001 0001 0001
0001 0001 0001

Button 6 pressed

0001 0001 0001
0001 0001 0000
0001 0001 0001
0001 0001 0001


=== Access denied ===

Button released
```

Лістинг коду

```
#include "project.h"

#define PASS_LEN 3
const uint8_t PASSWORD[PASS_LEN] = {1, 2, 3};

uint8_t input_buffer[PASS_LEN];
uint8_t input_index = 0;

void setLEDColor(uint8_t r, uint8_t g, uint8_t b) {
    RED_Write(r);
    GREEN_Write(g);
    BLUE_Write(b);
}

static void (*COLUMN_X_SetDriveMode[3]) (uint8_t mode) = {
    COLUMN_0_SetDriveMode,
    COLUMN_1_SetDriveMode,
    COLUMN_2_SetDriveMode
};

static void (*COLUMN_x_Write[3]) (uint8_t value) = {
    COLUMN_0_Write,
    COLUMN_1_Write,
    COLUMN_2_Write
};

static uint8 (*ROW_x_Read[4]) () = {
    ROW_0_Read,
    ROW_1_Read,
    ROW_2_Read,
    ROW_3_Read
};

static uint8_t keys[4][3];

static const uint8_t key_map[4][3] = {
    {1,  2,  3},
    {4,  5,  6},
    {7,  8,  9},
    {10, 11, 12}
};

static void initMatrix()
{
    for(int column_index = 0; column_index < 3; column_index++)
    {
        COLUMN_X_SetDriveMode[column_index](COLUMN_0_DM_DIG_HIZ);
    }
}

static void readMatrix()
{
    for(int column_index = 0; column_index < 3; column_index++)
    {
        COLUMN_X_SetDriveMode[column_index](COLUMN_0_DM_STRONG);
        COLUMN_x_Write[column_index](0);

        for(int row_index = 0; row_index < 4; row_index++)
        {
            keys[row_index][column_index] = ROW_x_Read[row_index]();
        }

        COLUMN_X_SetDriveMode[column_index](COLUMN_0_DM_DIG_HIZ);
    }
}

void printMatrix()
{
    SW_Tx_UART_PutCRLF();
    for (int row_index = 0; row_index < 4; row_index++)
    {
```

```
        for (int column_index = 0; column_index < 3; column_index++)
        {
            SW_Tx_UART_PutHexInt(keys[row_index][column_index]);
            SW_Tx_UART_PutString(" ");
        }
        SW_Tx_UART_PutCRLF();
    }
    SW_Tx_UART_PutCRLF();
}

int main(void)
{
    CyGlobalIntEnable;

    SW_Tx_UART_Start();
    CyDelay(50);

    SW_Tx_UART_PutCRLF();
    SW_Tx_UART_PutString("Software Transmit UART");
    SW_Tx_UART_PutCRLF();

    initMatrix();
    setLEDColor(0, 0, 0);

    uint8_t last_pressed = 0;
    uint8_t current_pressed = 0;

    for(;;)
    {
        readMatrix();
        current_pressed = 0;

        for (int r = 0; r < 4; r++)
        {
            for (int c = 0; c < 3; c++)
            {
                if (keys[r][c] == 0)
                {
                    current_pressed = key_map[r][c];
                }
            }
        }

        if (current_pressed != last_pressed)
        {
            if (current_pressed == 0)
            {
                SW_Tx_UART_PutString("Button released\r\n");
                setLEDColor(1, 1, 1);
                printMatrix();
            }
            else
            {
                switch (current_pressed)
                {
                    case 1:
                        SW_Tx_UART_PutString("Button 1 pressed\r\n");
                        setLEDColor(0, 1, 1);
                        break;
                    case 2:
                        SW_Tx_UART_PutString("Button 2 pressed\r\n");
                        setLEDColor(1, 0, 1);
                        break;
                    case 3:
                        SW_Tx_UART_PutString("Button 3 pressed\r\n");
                        setLEDColor(1, 1, 0);
                        break;
                    case 4:
                        SW_Tx_UART_PutString("Button 4 pressed\r\n");
                        setLEDColor(0, 0, 1);
                        break;
                    case 5:
                        SW_Tx_UART_PutString("Button 5 pressed\r\n");
                        setLEDColor(0, 1, 0);
```

```
                    break;
                case 6:
                    SW_Tx_UART_PutString("Button 6 pressed\r\n");
                    setLEDColor(1, 0, 0);
                    break;
                case 7:
                    SW_Tx_UART_PutString("Button 7 pressed\r\n");
                    setLEDColor(0, 1, 1);
                    break;
                case 8:
                    SW_Tx_UART_PutString("Button 8 pressed\r\n");
                    setLEDColor(1, 0, 1);
                    break;
                case 9:
                    SW_Tx_UART_PutString("Button 9 pressed\r\n");
                    setLEDColor(1, 1, 0);
                    break;
                case 10:
                    SW_Tx_UART_PutString("Button * pressed\r\n");
                    setLEDColor(0, 0, 1);
                    break;
                case 11:
                    SW_Tx_UART_PutString("Button 0 pressed\r\n");
                    setLEDColor(0, 1, 0);
                    break;
                case 12:
                    SW_Tx_UART_PutString("Button # pressed\r\n");
                    setLEDColor(1, 0, 0);
                    break;
            }
            printMatrix();

            /* ЛОГІКА ПАРОЛЯ */
            input_buffer[input_index] = current_pressed;
            input_index++;

            /* Якщо введено достатню кількість символів */
            if (input_index >= PASS_LEN)
            {
                uint8_t is_correct = 1;
                for (int i = 0; i < PASS_LEN; i++)
                {
                    if (input_buffer[i] != PASSWORD[i])
                    {
                        is_correct = 0;
                        break;
                    }
                }

                SW_Tx_UART_PutCRLF();
                if (is_correct == 1)
                {
                    SW_Tx_UART_PutString("=== Access allowed ===\r\n");
                }
                else
                {
                    SW_Tx_UART_PutString("=== Access denied ===\r\n");
                }
                SW_Tx_UART_PutCRLF();

                input_index = 0; /* Скидаємо лічильник для нового введення */
            }
        }
        last_pressed = current_pressed;
    }
    CyDelay(20);
  }
}
```

Висновки

Під час виконання лабораторної роботи, було ознайомлено з принципами підключення та використання матричної клавіатури для введення інформації у мікроконтролерних системах на базі PSoC. Набуті навички розробки проєкту в інтегрованому середовищі PSoC® CreatorTM, налаштування апаратних компонентів та програмної обробки введених даних.